

**Prototipo GESCAR.pdf**
PDF

Este es mi trabajo fin de grado, quiero utilizar xampp, python, visual studio code y que sea arquitectura cliente servidor. ¿que mas puedo necesitar?

Pensó durante un par de segundos >

Sí. Para tu prototipo de **Gestión de Carteras**, con pantallas de cartera, histórico, índices, alarmas, altas, exportación Excel e informe PDF, la combinación tiene sentido: **XAMPP + Python + VS Code + arquitectura cliente-servidor**.

 Prototipo GESCAR

Además necesitarías:

1. Backend servidor

- Python con **Flask** o **FastAPI**.
- Recomendación: **Flask**, más sencillo para TFG.
- Servirá las páginas, recibirá peticiones y consultará la base de datos.

2. Base de datos

- Con XAMPP usarías **MySQL/MariaDB**.
- Tablas mínimas:
 - usuarios
 - valores/acciones
 - cartera
 - operaciones
 - dividendos
 - alarmas
 - histórico de ventas

3. Frontend cliente

- HTML, CSS y JavaScript.
- Plantillas con **Jinja2** si usas Flask.
- Bootstrap para que las pantallas queden profesionales.

4. Librerías Python

- `flask`
- `flask-mysqldb` o `pymysql`
- `pandas` para exportar a Excel

- `openpyxl` para generar Excel
- `reportlab` o `fpdf2` para informes PDF
- `python-dotenv` para guardar configuración
- `werkzeug` para contraseñas seguras

5. Herramientas

- Visual Studio Code
- XAMPP: Apache + MySQL
- phpMyAdmin para ver la base de datos
- Git y GitHub para control de versiones
- Navegador web como cliente
- Postman, opcional, para probar endpoints

Arquitectura recomendada para tu TFG:

Cliente web → Flask/FastAPI → MySQL en XAMPP

Yo lo haría así:

```
gescar/  
|  
├─ app.py  
├─ config.py  
├─ requirements.txt  
|  
├─ templates/  
|   ├─ login.html  
|   ├─ menu.html  
|   ├─ cartera.html  
|   ├─ historico.html  
|   ├─ indices.html  
|   ├─ alarmas.html  
|   └─ alta.html  
|  
├─ static/  
|   ├─ css/  
|   ├─ js/  
|   └─ img/  
|  
├─ models/  
|   ├─ usuario.py  
|   ├─ cartera.py  
|   ├─ valor.py  
|   └─ alarma.py  
|  
└─ services/  
    └─ excel_service.py
```



```
└─ pdf_service.py
└─ cotizaciones_service.py
```

Para empezar, te diría que uses:

Python + Flask + MySQL/XAMPP + HTML/CSS/Bootstrap + JavaScript

Es suficiente, defendible para un TFG y no te complica demasiado.

Perfecto. A partir de aquí vamos a trabajar como lo haría un desarrollador profesional, pero explicando **el porqué** de cada cosa. Mi objetivo es que cuando termines el TFG sepas defenderlo delante del tribunal, no solo que funcione.

Paso 4: Crear un entorno virtual

Este es un concepto muy importante en Python.

¿Qué es un entorno virtual?

Imagina que en un ordenador tienes varios proyectos:

- Un proyecto usa Flask 3.0
- Otro usa Flask 2.2
- Otro usa Django

Si todos instalaran las librerías en el mismo sitio, acabarían interfiriendo unas con otras.

Por eso Python permite crear un **entorno virtual**.

Es una carpeta que contiene:

- el intérprete de Python
- las librerías de ese proyecto
- la configuración propia del proyecto

Así, cada proyecto es independiente.

¿Por qué lo vamos a usar en el TFG?

Porque es una buena práctica profesional.

Además, cuando entregues el proyecto bastará con decir:

```
"Instale las dependencias del fichero requirements.txt "
```

y cualquiera podrá ejecutarlo.

Vamos a crearlo

En la terminal de VS Code escribe:

```
</> Bash
```

```
python -m venv venv
```

¿Qué significa?

```
python
```

Ejecuta Python.

```
-m
```

Le dice a Python que ejecute un módulo.

```
venv
```



Es el módulo que crea entornos virtuales.

El último

```
venv
```



es simplemente el nombre de la carpeta.

Cuando termine verás una carpeta nueva:

```
GESCAR
```

```
|  
└─ venv
```



Activarlo

Después escribe:

```
<> Bash
```



```
venv\Scripts\activate
```

Si todo va bien, el principio de la línea cambiará a algo parecido a:

```
(venv) C:\Users\Alberto\Documents\GESCAR>
```



Ese `(venv)` significa que **todo lo que instalemos a partir de ahora quedará únicamente en este proyecto.**

¿Por qué no instalar Flask directamente?

Porque si mañana haces otro proyecto, no queremos mezclar sus librerías con las de GESCAR.

Es exactamente igual que cuando en Java cada proyecto tiene su propio `pom.xml` o `build.gradle`.

Tu turno

Haz estos dos comandos:

</> Bash



```
python -m venv venv
```

y luego

</> Bash



```
venv\Scripts\activate
```

y dime si ya ves el `(venv)` delante del prompt. Después pasaremos al siguiente paso: instalar Flask y entender qué es un servidor web.

lo veo

Perfecto.

Paso 5: instalar Flask

Con el `(venv)` activado, escribe:

</> Bash



```
pip install flask
```

¿Qué es Flask?

Flask es una librería de Python que nos permite crear un **servidor web**.

Es decir:

El usuario abre una URL

↓

Flask recibe la petición

↓

Python decide qué responder

↓

El navegador muestra una página



Por ejemplo, cuando más adelante entres en:

http://localhost:5000



Flask será quien responda con la pantalla principal de **Gestión de Carteras**.

Cuando termine la instalación, dime qué te sale o si ves algún error.

todo correcto

Genial.

Paso 6: crear el primer servidor

En VS Code, crea un archivo en la carpeta **GESCAR** llamado:

app.py



Dentro pega esto:

<> Python



Ejecutar

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Servidor GESCAR funcionando correctamente"

if __name__ == "__main__":
    app.run(debug=True)
```

¿Qué hace este código?

`Flask(__name__)` crea la aplicación web.

`@app.route("/")` dice qué debe pasar cuando el usuario entra en la página principal.

`def inicio()` es la función que responde.

`app.run(debug=True)` arranca el servidor en modo desarrollo.

Ahora ejecútalo en la terminal:

</> Bash



```
python app.py
```

Después abre en el navegador:

```
http://localhost:5000
```



Deberías ver:

```
Servidor GESCAR funcionando correctamente
```



Dime si te aparece.

si

¡Perfecto! 🎉 Ya has creado tu **primer servidor cliente-servidor en Python**. Aunque ahora parezca muy simple, acabas de dar el paso más importante del proyecto.

Quiero detenerme un momento para explicarte qué ha pasado realmente.

¿Qué acaba de ocurrir?

Cuando has ejecutado:

</> Bash



```
python app.py
```

Python ha arrancado un programa que está **escuchando** peticiones en el puerto **5000**.

El servidor queda esperando:

```
Esperando...  
Cliente -----> Servidor Flask  
"Quiero la página /"
```



Cuando el navegador solicita:

```
http://localhost:5000/
```



envía una petición HTTP.

Flask la recibe:

GET /



Busca si existe una ruta para "/".

La encuentra aquí:

</> Python



▶ Ejecutar

```
@app.route("/")  
def inicio():
```

Entonces ejecuta la función:

</> Python



▶ Ejecutar

```
return "Servidor GESCAR funcionando correctamente"
```

y envía esa respuesta al navegador.

Esto YA es arquitectura cliente-servidor

CLIENTE

Google Chrome



HTTP



SERVIDOR

Python + Flask



(De momento nada)



Más adelante será:

Navegador



Servidor Flask



Y esa será exactamente la arquitectura que podrás explicar en la memoria del TFG.

Vamos a hacerlo como un proyecto profesional

Hasta ahora todo está en un único archivo:

app.py



Eso sirve para aprender, pero un proyecto real no se organiza así.

Nuestro objetivo será tener esta estructura

```
GESCAR
|
├─ app.py           ← Punto de entrada
|
├─ config.py        ← Configuración
|
├─ requirements.txt ← Librerías del proyecto
|
├─ routes/          ← Todas las rutas
|   │ inicio.py
|   │ cartera.py
|   │ alarmas.py
|   │ historico.py
|
├─ models/          ← Acceso a MySQL
|
├─ services/        ← Lógica de negocio
|
├─ templates/       ← HTML
|
├─ static/
|   │ css/
|   │ js/
|   │ img/
|
└─ database/
    │ scripts.sql
```



Esta estructura es muy similar a la que utilizan muchas aplicaciones Flask profesionales y demuestra una buena organización del código.

El siguiente paso

Ahora podríamos conectar MySQL directamente, pero **no lo vamos a hacer todavía**.

¿Por qué?

Porque antes necesitamos entender cómo funciona Flask mostrando páginas HTML. Si conectáramos la base de datos ahora, estaríamos mezclando dos conceptos nuevos a la vez.

El siguiente paso será crear la **primera pantalla del proyecto**, sustituyendo el texto plano por una página web HTML. A partir de ahí empezaremos a construir la interfaz de tu prototipo (menú, cartera, histórico, etc.) siguiendo el diseño del PDF.  Prototipo GESCAR

Mi propuesta para el desarrollo del TFG

Vamos a construir el proyecto en este orden:

1.  Servidor Flask (hecho).
2. Crear la primera página HTML.
3. Aprender cómo Flask envía páginas al navegador.
4. Añadir estilos CSS y Bootstrap.
5. Diseñar el menú principal igual al prototipo.
6. Conectar con MySQL.
7. Implementar el login.
8. Desarrollar cada módulo (Mi Cartera, Histórico, Índices, Alarmas y Altas).
9. Exportación a Excel y generación de PDF.
10. Mejoras finales y preparación de la memoria y la defensa.

Creo que esta forma es la mejor porque en cada paso aprenderás un concepto nuevo sin sentir que el proyecto se vuelve demasiado complejo de golpe.

seguimos

Seguimos.

Paso 7: crear la primera página HTML

Primero para el servidor con:

```
</> Bash
```



```
CTRL + C
```

Ahora crea una carpeta llamada:

templates



Dentro de `templates`, crea un archivo:

inicio.html



La estructura debe quedar así:

```
GESCAR
├─ app.py
├─ venv
└─ templates
    └─ inicio.html
```



Ahora cambia `app.py` por esto:

</> Python



▶ Ejecutar

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def inicio():
    return render_template("inicio.html")

if __name__ == "__main__":
    app.run(debug=True)
```

Y en `templates/inicio.html` pega esto:

HTML



```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>GESCAR</title>
</head>
<body>
  <h1>Gestión de Carteras</h1>
  <p>Aplicación cliente-servidor con Python, Flask y MySQL.</p>
</body>
</html>
```

¿Qué hemos hecho?

Antes Flask devolvía texto:

</> Python



Ejecutar

```
return "Servidor GESCAR funcionando correctamente"
```

Ahora devuelve una página HTML:

</> Python



Ejecutar

```
return render_template("inicio.html")
```

Flask buscará automáticamente el archivo dentro de la carpeta `templates`.

Ahora ejecuta otra vez:

</> Bash



```
python app.py
```

Y abre:

```
http://localhost:5000
```



Deberías ver el título **Gestión de Carteras**.

correcto

Perfecto. Hasta ahora has aprendido dos cosas fundamentales:

- Un servidor puede devolver **texto**.
- Un servidor puede devolver una **página web**.

Ahora viene un concepto muy importante que utilizarás durante todo el proyecto.

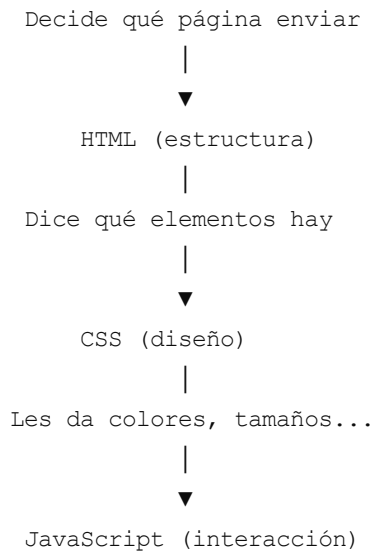
Paso 8: Entender HTML, CSS y Flask

Muchos principiantes piensan que Flask "hace las pantallas", pero no es así.

Cada tecnología tiene una función distinta.

FLASK (Python)





Por eso es importante separar responsabilidades.

Vamos a crear una página profesional

Paso 1

Crea una carpeta llamada:

static

Dentro crea otra:

css

Debe quedar así:

```
GESCAR
|
├─ static
|   └─ css
|       └─ estilos.css
└─ templates
    └─ inicio.html
```

Ahora crea el archivo

estilos.css

Paso 2

Dentro escribe:

</> CSS



```
body{
  margin:0;
  font-family: Arial, Helvetica, sans-serif;
  background:#f5f5f5;
}

header{
  background:#003366;
  color:white;
  padding:20px;
  text-align:center;
}

main{
  padding:40px;
}

.tarjeta{
  background:white;
  padding:20px;
  border-radius:10px;
  box-shadow:0 0 10px rgba(0,0,0,.2);
}
```

¿Qué significa?

body

Es todo el documento.

</> CSS



```
background:#f5f5f5;
```

pone el fondo gris claro.

header

Será la cabecera.

</> CSS



```
background:#003366;
```

azul oscuro.

.tarjeta

El punto significa:

"esta clase puede utilizarse en cualquier elemento".

Más adelante tendremos muchas tarjetas.

Paso 3

Ahora modifica `inicio.html`

HTML



```
<!DOCTYPE html>
<html lang="es">

<head>

  <meta charset="UTF-8">

  <title>GESCAR</title>

  <link rel="stylesheet"
        href="{ url_for('static', filename='css/estilos.css') }">

</head>

<body>

<header>
```

Vamos a entender esta línea

Es probablemente la más rara que has visto hasta ahora.

HTML



```
<link rel="stylesheet"
      href="{{ url_for('static', filename='css/estilos.css') }}">
```

No quiero que la copies sin entenderla.

Flask utiliza un motor de plantillas llamado **Jinja2**.

Cuando ve:

HTML



```
{{ ... }}
```

sabe que ahí debe ejecutar código Python antes de enviar la página.

En concreto:

Python



Ejecutar

```
url_for("static", filename="css/estilos.css")
```

genera automáticamente la dirección correcta del archivo CSS.

El navegador acaba recibiendo algo parecido a:

HTML



```
<link rel="stylesheet" href="/static/css/estilos.css">
```

¿Por qué hacerlo así y no escribir directamente `/static/css/estilos.css` ? Porque si algún día cambia la configuración de la aplicación, Flask seguirá generando la ruta correcta sin que tengas que modificar el HTML. Es una buena práctica que verás en muchos proyectos profesionales.

Una pequeña reflexión

A partir de este momento quiero que el proyecto no sea solo "hacer pantallas". Quiero que entiendas **por qué** hacemos cada cosa. Esa comprensión te ayudará mucho en la defensa del TFG.

Además, a partir del siguiente paso empezaremos a construir una aplicación con una arquitectura muy similar a la que se utiliza en entornos profesionales: separaremos el

código por módulos (rutas, acceso a datos, lógica de negocio, plantillas, etc.) en lugar de dejar todo dentro de `app.py`.

Cuando tengas esta página con el estilo aplicado (cabecera azul, fondo gris y una tarjeta blanca), me lo dices y comenzaremos a reorganizar el proyecto antes de añadir la base de datos. Ese cambio marcará el paso de un ejemplo sencillo a una aplicación bien estructurada.

ya está

Perfecto. Ahora es donde quiero que empecemos a trabajar **como ingenieros de software** y no simplemente como programadores.

Hasta ahora hemos creado una aplicación pequeña. A partir de este punto vamos a construir un proyecto que, si mañana una empresa lo viera, diría: *"está bien estructurado"*.

Paso 9: Diseñar la arquitectura del proyecto

Antes de escribir más código, quiero explicarte un concepto muy importante.

¿Qué es la arquitectura de una aplicación?

Imagina un restaurante.

¿El cocinero atiende las mesas?

No.

¿El camarero cocina?

No.

¿El cajero prepara los postres?

Tampoco.

Cada uno tiene una responsabilidad.

En programación ocurre exactamente lo mismo.

¿Qué tenemos ahora?

Todo está prácticamente en un único sitio.

Cliente



app.py



HTML



Funciona.

Pero cuando tengamos:

- Login
- Usuarios
- Cartera
- Histórico
- Alarmas
- Exportación Excel
- PDF
- Índices

`app.py` tendría miles de líneas.

Sería muy difícil de mantener.

La solución: separar responsabilidades

Vamos a dividir el proyecto.

Cliente



app.py



Routes

Services

Models



Templates

Lógica

Base de datos



Cada carpeta tendrá un trabajo.

Solo arrancará Flask.

Nada más.

2 routes

Aquí estarán las pantallas.

Por ejemplo:

```
routes/
```



contendrá

```
inicio.py  
cartera.py  
historico.py  
alarmas.py  
usuarios.py
```



Cada archivo tendrá únicamente las rutas de esa parte.

3 models

Aquí irá todo lo relacionado con MySQL.

Ejemplo:

```
</> Python
```



Ejecutar

```
obtener_usuario()  
  
guardar_operacion()  
  
listar_cartera()
```

Nunca habrá HTML aquí.

4 services

Aquí irá la lógica del negocio.

Por ejemplo:

Calcular la rentabilidad.

```
Compra = 10€
```

```
Actual = 13€
```

```
Rentabilidad = 30%
```



Eso no pertenece ni a HTML ni a MySQL.

Pertenece al negocio.

5 templates

Las páginas HTML.

6 static

CSS

JavaScript

Imágenes

¿Por qué esto gusta tanto en un TFG?

Porque demuestra que conoces el patrón de separación de responsabilidades.

Podrás decir en la defensa:

"He separado la presentación, la lógica de negocio y el acceso a datos para facilitar el mantenimiento y la escalabilidad de la aplicación."

Eso suele gustar mucho al tribunal.

Vamos a reorganizar el proyecto

Crea estas carpetas:

```
routes
```



models



services



database



Debe quedar algo parecido a esto:

GESCAR



```
|
├─ app.py
├─ config.py
├─ requirements.txt
|
├─ database
|
├─ models
|
├─ routes
|
├─ services
|
├─ static
│   └─ css
|
├─ templates
|
└─ venv
```

¿Por qué también vamos a crear `config.py`?

Porque **nunca** debemos poner datos importantes (como la contraseña de la base de datos) repartidos por el código.

En un único archivo centralizaremos toda la configuración:

- Puerto del servidor.
- Nombre de la base de datos.
- Usuario de MySQL.
- Contraseña.
- Clave secreta de Flask.

Así, si algún día cambias la contraseña de MySQL, solo tendrás que modificar **una línea**.

¿Por qué quiero hacer esto antes de conectar MySQL?

Porque he visto muchos TFG donde primero conectan la base de datos y luego intentan reorganizar el proyecto. El resultado suele ser un código difícil de mantener.

Nosotros vamos a construir la aplicación como si fuéramos un equipo profesional: primero una buena estructura y luego iremos añadiendo funcionalidades.

Una propuesta para que el TFG quede excelente

Además de ayudarte a programarlo, me gustaría que siguiéramos estas dos normas durante todo el desarrollo:

- 1. No copiar código sin entenderlo.** Cada vez que añadamos algo, te explicaré qué hace y por qué se hace.
- 2. Ir pensando en la defensa del TFG.** Te iré indicando qué conceptos merece la pena mencionar en la memoria y qué preguntas podría hacerte el tribunal.

De esta forma, cuando terminemos no solo tendrás una aplicación funcionando, sino que sabrás explicar cada decisión técnica que has tomado. Creo que eso marcará una diferencia importante el día de la presentación.

seguimos

Seguimos.

Paso 10: crear los archivos de arquitectura

Dentro de las carpetas que has creado, crea estos archivos:

```
routes/inicio_routes.py
```



```
routes/__init__.py
```



```
models/__init__.py
```



```
services/__init__.py
```



Y en la raíz del proyecto:

```
config.py
```



La estructura queda así:

```
GESCAR
├─ app.py
├─ config.py
├─ routes
│   ├── __init__.py
│   └─ inicio_routes.py
├─ models
│   └─ __init__.py
├─ services
│   └─ __init__.py
├─ templates
│   └─ inicio.html
└─ static
    └─ css
        └─ estilos.css
```

¿Qué es `__init__.py`?

Es un archivo que le dice a Python:

“Esta carpeta puede tratarse como un paquete/importable”.

Aunque esté vacío, nos permite luego hacer imports como:

<> Python



Ejecutar

```
from routes.inicio_routes import inicio_bp
```

Paso 11: mover la ruta de inicio

Ahora abre `routes/inicio_routes.py` y pega:

<> Python



Ejecutar

```
from flask import Blueprint, render_template

inicio_bp = Blueprint("inicio", __name__)

@inicio_bp.route("/")
def inicio():
    return render_template("inicio.html")
```

¿Qué es `Blueprint`?

Un `Blueprint` es una forma de separar rutas en Flask.

En vez de tener todas las pantallas en `app.py`, las agrupamos por módulos:

```
inicio_routes.py    → pantalla de inicio
cartera_routes.py   → pantalla de cartera
alarmas_routes.py   → pantalla de alarmas
```



Paso 12: modificar `app.py`

Sustituye todo el contenido de `app.py` por esto:

</> Python



▶ Ejecutar

```
from flask import Flask
from routes.inicio_routes import inicio_bp

app = Flask(__name__)

app.register_blueprint(inicio_bp)

if __name__ == "__main__":
    app.run(debug=True)
```

¿Qué hemos hecho?

Ahora `app.py` ya no contiene la ruta `/`.

Solo hace tres cosas:

1. Crear la aplicación Flask
2. Registrar las rutas
3. Arrancar el servidor



Eso es mucho más limpio.

Paso 13: probar

Ejecuta otra vez:

</> Bash



```
python app.py
```

Y abre:

```
http://localhost:5000
```



Tiene que verse igual que antes.

Dime si funciona.

funciona

Perfecto.

Paso 14: guardar las librerías del proyecto

En la terminal, con `(venv)` activado, ejecuta:

`</> Bash`



```
pip freeze > requirements.txt
```

¿Qué hace?

Crea un archivo con las librerías instaladas, por ejemplo Flask.

Sirve para que otra persona pueda instalar lo mismo con:

`</> Bash`



```
pip install -r requirements.txt
```

Es importante para entregar el TFG.

Cuando lo tengas, seguimos con el **menú principal de GESCAR**.

seguimos

Paso 15: crear el menú principal

Tu prototipo tiene un menú con estas opciones:

```
Mi cartera  
Histórico  
Índices  
Alarmas  
Altas  
Salir
```



Vamos a crear esa primera pantalla.

Abre `templates/inicio.html` y sustitúyelo por esto:

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>GESCAR - Menú</title>

  <link rel="stylesheet"
        href="{{ url_for('static', filename='css/estilos.css') }}">
</head>

<body>

<header>
  <h1>Gestión de Carteras</h1>
  <p>Usuario: User1</p>
</header>

<main class="menu-principal">
```

Ahora abre `static/css/estilos.css` y añade al final:

CSS

```
.menu-principal{
  display: flex;
  flex-direction: column;
  gap: 20px;
  max-width: 400px;
  margin: 40px auto;
}

.opcion-menu{
  display: block;
  background: white;
  padding: 22px;
  text-align: center;
  text-decoration: none;
  color: #003366;
  font-size: 22px;
  font-weight: bold;
  border-radius: 12px;
  box-shadow: 0 0 10px rgba(0,0,0,.15);
}

.opcion-menu:hover{
  background: #003366;
  color: white;
}
```

```
}  
  
.salir{  
  color: #990000;  
}
```

¿Qué estamos haciendo?

En el HTML creamos los botones del menú con etiquetas `<a>`, que son enlaces.

En el CSS usamos:

`</>` CSS



```
display: flex;  
flex-direction: column;  
gap: 20px;
```

para colocar las opciones una debajo de otra con separación.

Ahora guarda los archivos y recarga:

`http://localhost:5000`



Deberías ver el menú principal parecido al prototipo.

siguiente

Pensó durante un segundo >

Paso 16: crear las rutas del menú

Ahora los botones no deben apuntar a `#`, sino a páginas reales.

Crea estos archivos dentro de `routes`:

```
cartera_routes.py  
historico_routes.py  
indices_routes.py  
alarmas_routes.py  
altas_routes.py
```



En `routes/cartera_routes.py` pega:

</> Python



▶ Ejecutar

```
from flask import Blueprint, render_template

cartera_bp = Blueprint("cartera", __name__)

@cartera_bp.route("/cartera")
def cartera():
    return render_template("cartera.html")
```

En `historico_routes.py`:

</> Python



▶ Ejecutar

```
from flask import Blueprint, render_template

historico_bp = Blueprint("historico", __name__)

@historico_bp.route("/historico")
def historico():
    return render_template("historico.html")
```

En `indices_routes.py`:

</> Python



▶ Ejecutar

```
from flask import Blueprint, render_template

indices_bp = Blueprint("indices", __name__)

@indices_bp.route("/indices")
def indices():
    return render_template("indices.html")
```

En `alarmas_routes.py`:

</> Python



▶ Ejecutar

```
from flask import Blueprint, render_template

alarmas_bp = Blueprint("alarmas", __name__)

@alarmas_bp.route("/alarmas")
def alarmas():
    return render_template("alarmas.html")
```

En `altas_routes.py`:

</> Python



▶ Ejecutar

```
from flask import Blueprint, render_template

altas_bp = Blueprint("altas", __name__)

@altas_bp.route("/altas")
def altas():
    return render_template("altas.html")
```

Paso 17: registrar esas rutas en `app.py`

Modifica `app.py` así:

</> Python



▶ Ejecutar

```
from flask import Flask

from routes.inicio_routes import inicio_bp
from routes.cartera_routes import cartera_bp
from routes.historico_routes import historico_bp
from routes.indices_routes import indices_bp
from routes.alarmas_routes import alarmas_bp
from routes.altas_routes import altas_bp

app = Flask(__name__)

app.register_blueprint(inicio_bp)
app.register_blueprint(cartera_bp)
app.register_blueprint(historico_bp)
app.register_blueprint(indices_bp)
app.register_blueprint(alarmas_bp)
app.register_blueprint(altas_bp)

if __name__ == "__main__":
    app.run(debug=True)
```

Paso 18: crear las páginas HTML

Dentro de `templates`, crea:

```
cartera.html
historico.html
indices.html
alarmas.html
altas.html
```



Por ahora, en cada una pon algo simple. Ejemplo para `cartera.html`:

HTML



```
<h1>Mi Cartera</h1>
<a href="/">Volver al menú</a>
```

Haz lo mismo cambiando el título en cada archivo.

Paso 19: enlazar el menú

En `inicio.html`, cambia los `href="#"` por:

HTML



```
<a class="opcion-menu" href="/cartera">👛 MI CARTERA</a>
<a class="opcion-menu" href="/historico">📖 HISTÓRICO</a>
<a class="opcion-menu" href="/indices">📊 ÍNDICES</a>
<a class="opcion-menu" href="/alarmas">🚨 ALARMAS</a>
<a class="opcion-menu" href="/altas">📝 ALTAS</a>
<a class="opcion-menu salir" href="#">SALIR</a>
```

Pruébalo. Al pulsar cada botón ya debería abrir su pantalla.

funciona, siguiente

Pensó durante un par de segundos >

Perfecto.

Paso 20: crear una plantilla base

Ahora tenemos varias páginas, pero no queremos repetir siempre cabecera, CSS, estructura, etc.

Crea en `templates` un archivo llamado:

base.html



Pega esto:

HTML



```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}GESCAR{% endblock %}</title>

  <link rel="stylesheet"
```

```
        href="{{ url_for('static', filename='css/estilos.css') }}">

</head>

<body>

<header>
    <h1>Gestión de Carteras</h1>
    <p>Usuario: User1</p>
</header>
```

¿Qué es esto?

Es una **plantilla madre**.

La parte importante es esta:

 HTML



```
{% block content %}
{% endblock %}
```

Significa:

“Aquí cada página meterá su contenido propio”.

Así evitamos repetir todo el HTML en cada archivo.

Paso 21: modificar `cartera.html`

Cambia `cartera.html` por esto:

 HTML



```
{% extends "base.html" %}

{% block title %}GESCAR - Mi Cartera{% endblock %}

{% block content %}
    <div class="tarjeta">
        <h2>Mi Cartera</h2>
        <p>Aquí mostraremos las acciones actualmente en cartera.</p>

        <a href="/">Volver al menú</a>
    </div>
{% endblock %}
```

¿Qué significa?

HTML



```
{% extends "base.html" %}
```

quiere decir:

“Esta página usa la estructura de `base.html`”.

Y esto:

HTML



```
{% block content %}
```

rellena el hueco que dejamos en la plantilla base.

Haz lo mismo con las demás páginas cambiando solo el título.

Por ejemplo:

HTML



```
<h2>Histórico</h2>
```

HTML



```
<h2>Índices</h2>
```

HTML



```
<h2>Alarmas</h2>
```

HTML



```
<h2>Altas</h2>
```

Cuando lo tengas, recarga y comprueba que siguen funcionando todas.

funciona, siguiente

Paso 22: preparar la base de datos en XAMPP

Ahora vamos a crear la base de datos MySQL.

Abre en el navegador:

`http://localhost/phpmyadmin`



¿Qué es phpMyAdmin?

Es una herramienta web que viene con XAMPP y permite gestionar MySQL sin escribir todo por comandos.

Ahí vamos a crear la base de datos de GESCAR.

Paso 23: crear la base de datos

En phpMyAdmin:

1. Pulsa **Nueva**.
2. En “Nombre de la base de datos” escribe:

`gescar_db`



3. En cotejamiento selecciona, si aparece:

`utf8mb4_spanish_ci`



4. Pulsa **Crear**.

¿Por qué creamos una base de datos?

Porque necesitamos guardar información permanente:

`usuarios`
`acciones`
`compras`
`ventas`
`dividendos`
`alarmas`



Si lo guardáramos solo en Python, al cerrar el programa se perdería.

Cuando tengas creada `gescar_db`, dime **creada**.

creada

Perfecto.

Paso 24: crear las tablas

En phpMyAdmin, entra en la base de datos:

gescar_db



Pulsa la pestaña **SQL** y pega esto:

</> SQL



```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    usuario VARCHAR(50) NOT NULL UNIQUE,  
    email VARCHAR(100),  
    telefono VARCHAR(20),  
    password VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE valores (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    ticker VARCHAR(10) NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    mercado VARCHAR(20),  
    cotizacion_actual DECIMAL(10,2) DEFAULT 0  
);  
  
CREATE TABLE operaciones (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    usuario_id INT NOT NULL,  
    valor_id INT NOT NULL,  
    tipo ENUM('COMPRA', 'VENTA') NOT NULL,  
    numero_acciones INT NOT NULL,  
    precio DECIMAL(10,2) NOT NULL,  
    gastos DECIMAL(10,2) DEFAULT 0,  
    fecha DATE NOT NULL,  
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id),  
    FOREIGN KEY (valor_id) REFERENCES valores(id)  
);
```

Pulsa **Continuar**.

¿Qué estamos creando?

- `usuarios`: los usuarios de la aplicación.
- `valores`: las acciones, por ejemplo TEF, CABK, IDR.
- `operaciones`: compras y ventas.

Cuando lo tengas, dime **tablas creadas**.

tablas creadas

Perfecto.

Paso 25: meter datos de prueba

En phpMyAdmin, pestaña **SQL**, pega esto:

</> SQL



```
INSERT INTO usuarios (nombre, usuario, email, telefono, password)
VALUES ('User Fernández', 'User1', 'user1@user1.com', '666554433', '1234');

INSERT INTO valores (ticker, nombre, mercado, cotizacion_actual)
VALUES
('TEF', 'Telefónica', 'BME', 3.90),
('CABK', 'CaixaBank', 'BME', 12.50),
('IDR', 'Indra', 'BME', 51.00);

INSERT INTO operaciones (usuario_id, valor_id, tipo, numero_acciones, precio, gastos, fecha)
VALUES
(1, 1, 'COMPRA', 150, 3.50, 11.00, '2005-12-12'),
(1, 2, 'COMPRA', 200, 10.50, 11.00, '2026-03-19'),
(1, 3, 'COMPRA', 200, 50.00, 11.00, '2005-01-01');
```

¿Para qué sirve?

Son datos parecidos a los de tu prototipo:

- TEF Telefónica
- CABK CaixaBank
- IDR Indra
- User1

Así podremos construir la pantalla **Mi Cartera** con datos reales de MySQL.

Cuando lo tengas, dime **datos insertados**.

datos insertados

¡Muy bien! Hasta aquí ya has conseguido algo importante:

- ☒ Un servidor Flask funcionando.

- ☒ Una arquitectura organizada por módulos.
- ☒ Varias pantallas conectadas entre sí.
- ☒ Una base de datos MySQL.
- ☒ Datos reales almacenados.

A partir de ahora empieza la parte más interesante: **conectar Python con MySQL**.

Antes de seguir, quiero hacer una pequeña parada

Hasta ahora hemos ido bastante rápido porque estábamos preparando el terreno. A partir de este punto **vamos a bajar el ritmo**.

¿Por qué?

Porque ahora empiezan conceptos importantes:

- Conexión a la base de datos.
- Consultas SQL desde Python.
- Separación entre la capa de acceso a datos y la interfaz.
- Patrón MVC (Modelo-Vista-Controlador).

Estos conceptos son los que realmente diferencian un TFG "correcto" de uno muy bueno.

Una mejora que quiero hacer respecto a lo que hemos construido

Hay una decisión que tomaría diferente antes de seguir.

Hasta ahora hemos creado:

```
routes/  
models/  
services/
```



Está bien.

Pero para un proyecto que va a crecer bastante (y el tuyo lo hará), yo añadiría una carpeta más:

```
database/
```



Y dentro:

```
database/
|
├─ conexion.py
├─ schema.sql
└─ datos_prueba.sql
```



¿Por qué?

Porque así separas claramente:

- `database/` → todo lo relacionado con MySQL.
- `models/` → clases que representan las entidades del negocio (Usuario, Operación, Valor...).

Esta separación es muy habitual en proyectos profesionales.

También vamos a dejar de escribir SQL dentro del código

Muchos tutoriales hacen esto:

```
<> Python
```



Ejecutar

```
cursor.execute("SELECT * FROM usuarios")
```

Funciona.

Pero dentro de seis meses tendrás cientos de consultas mezcladas con el resto del código.

Yo prefiero que desde el principio trabajemos así:

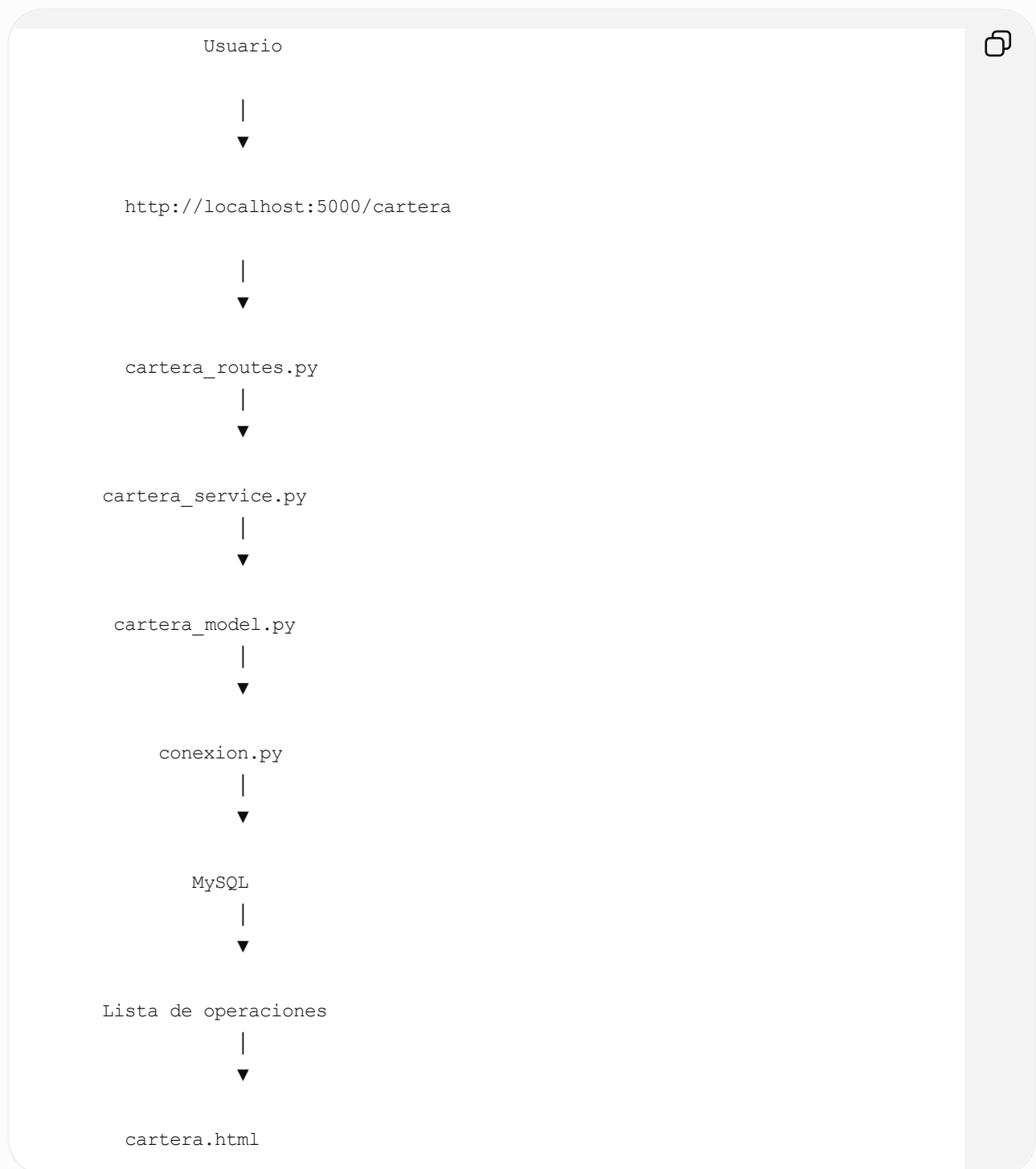
```
routes
  ↓
services
  ↓
models
  ↓
database
```



Es un poco más de trabajo ahora, pero el proyecto quedará muchísimo más limpio y será más fácil de explicar en la memoria.

Lo que vamos a construir

Dentro de unas horas de trabajo, cuando pulses **Mi Cartera**, ocurrirá exactamente esto:



Y esto, Alberto, **es arquitectura cliente-servidor de verdad**. Es el tipo de diagrama que puedes incluir en tu memoria y explicar con confianza en la defensa.

Mi propuesta

A partir de aquí quiero que dejemos de "hacer una web" y empecemos a desarrollar una **aplicación profesional**.

Eso significa que cada paso tendrá tres partes:

1. **La teoría** (qué problema estamos resolviendo).
2. **La implementación** (el código).
3. **La explicación** (por qué lo hemos hecho así y no de otra manera).

Así, cuando el tribunal te pregunte:

"¿Por qué ha separado la lógica de negocio del acceso a datos?"

No responderás "porque ChatGPT me dijo que lo hiciera", sino que entenderás perfectamente el motivo.

Antes de continuar: una sugerencia importante

He visto muchos TFG y creo que el tuyo puede quedar muy por encima de la media si hacemos una cosa más.

En lugar de limitarnos a "que funcione", vamos a desarrollar la aplicación siguiendo una arquitectura inspirada en **MVC (Modelo-Vista-Controlador)** y aplicando buenas prácticas (capas, reutilización de código, configuración centralizada, etc.).

Es un poco más de trabajo, pero:

- la aplicación será más fácil de ampliar,
- la memoria tendrá mucho más contenido técnico,
- y la defensa será mucho más sólida.

Creo sinceramente que merece la pena hacerlo así. El tiempo invertido ahora te ahorrará muchos problemas cuando el proyecto crezca y, además, te servirá como una excelente carta de presentación si algún día enseñas este proyecto en una entrevista de trabajo.

siguiente paso

Paso 26: instalar el conector de MySQL

Para que Python pueda hablar con MySQL necesitamos una librería.

En la terminal, con `(venv)` activado, ejecuta:

 Bash



```
pip install pymysql
```

¿Qué es pymysql ?

Es el “traductor” entre Python y MySQL.

```
Python/Flask → PyMySQL → MySQL/XAMPP
```



Sin esta librería, Flask puede mostrar páginas, pero no puede consultar la base de datos.

Cuando termine, actualiza también `requirements.txt` :

</> Bash



```
pip freeze > requirements.txt
```

Dime cuando lo tengas y seguimos creando `database/conexion.py` .

creado

Paso 27: crear la conexión con MySQL

Dentro de la carpeta `database` , crea este archivo:

```
conexion.py
```



Pega esto:

</> Python



Ejecutar

```
import pymysql

def obtener_conexion():
    return pymysql.connect (
        host="localhost",
        user="root",
        password="",
        database="gescar_db",
        cursorclass=pymysql.cursors.DictCursor
    )
```

¿Qué hace?

Esta función abre una conexión con MySQL.

</> Python



Ejecutar

```
host="localhost"
```

La base de datos está en tu propio ordenador.

</> Python



Ejecutar

```
user="root"  
password=""
```

En XAMPP normalmente el usuario es `root` y la contraseña está vacía.

</> Python



Ejecutar

```
database="gescar_db"
```

Es la base de datos que has creado.

</> Python



Ejecutar

```
DictCursor
```

Hace que MySQL devuelva los datos como diccionarios, más cómodo:

</> Python



Ejecutar

```
{"ticker": "TEF", "nombre": "Telefónica"}
```

Cuando lo tengas, dime **hecho**.

hecho

Perfecto. Ahora viene uno de los pasos más importantes de todo el proyecto. Aquí empezamos a aplicar una arquitectura profesional.

Paso 28: Crear el primer modelo

Antes de programar: ¿qué es un modelo?

En programación, un **modelo** es la parte encargada de hablar con la base de datos.

Piensa en esta situación:

El usuario pulsa **"Mi Cartera"**.

¿Quién debe ir a buscar los datos a MySQL?

-  `cartera.html` → No, solo muestra información.
-  `cartera_routes.py` → No, solo recibe la petición del navegador.
-  `cartera_model.py` → Sí, su trabajo es consultar la base de datos.

Por eso existe la carpeta **models**.

Vamos a crear nuestro primer modelo

Crea el archivo:

`models/cartera_model.py`



Y escribe este código:

 Python



 Ejecutar

```
from database.conexion import obtener_conexion

def obtener_operaciones():
    conexion = obtener_conexion()

    try:
        with conexion.cursor() as cursor:

            sql = """
                SELECT
                    o.id,
                    v.ticker,
                    v.nombre,
                    o.numero_acciones,
                    o.precio,
                    o.gastos,
                    o.fecha
                FROM operaciones o
                INNER JOIN valores v
                    ON o.valor_id = v.id
                ORDER BY o.fecha
            """

            cursor.execute(sql)

            operaciones = cursor.fetchall()
```

```
        return operaciones

    finally:
        conexion.close()
```

Vamos a entender este código línea por línea

1.

</> Python



Ejecutar

```
from database.conexion import obtener_conexion
```

Importamos la función que hicimos antes.

No sabemos cómo conecta a MySQL.

Y no queremos saberlo.

Solo sabemos que existe una función llamada:

</> Python



Ejecutar

```
obtener_conexion()
```

Eso es una ventaja enorme.

Si mañana cambias de MySQL a PostgreSQL, el resto del proyecto apenas cambiará.

2.

</> Python



Ejecutar

```
conexion = obtener_conexion()
```

Abre una conexión.

Es como descolgar el teléfono para llamar a MySQL.

3.

</> Python



▶ Ejecutar

```
with conexion.cursor() as cursor:
```

El cursor es quien ejecuta SQL.

Puedes imaginarlo así:

Python

|

Cursor

|

MySQL



4.

</> Python



▶ Ejecutar

```
sql = """  
...  
"""
```

Guardamos la consulta SQL.

He preferido escribirla en varias líneas porque:

- se lee mejor
- podrás modificarla fácilmente

5.

</> SQL



```
SELECT
```

Pedimos únicamente los campos que necesitamos.

No hacemos:

</> SQL



```
SELECT *
```

¿Por qué?

Porque en proyectos grandes es una mala práctica.

Es mejor indicar exactamente qué columnas queremos.

6.

</> SQL



INNER JOIN

Aquí aparece uno de los conceptos más importantes de bases de datos.

Tenemos dos tablas:

valores

id	ticker	nombre
1	TEF	Telefónica

operaciones

id	valor_id	precio
1	1	3.50

La consulta hace:

```
operaciones.valor_id = valores.id
```



Y obtiene:

Ticker	Nombre	Precio
TEF	Telefónica	3.50

Es decir:

"Une las dos tablas para traer toda la información en una sola consulta."

7.

</> Python



▶ Ejecutar

```
cursor.execute(sql)
```

Ejecuta la consulta.

8.

</> Python



▶ Ejecutar

```
cursor.fetchall()
```

Recoge **todas** las filas.

Obtendremos algo parecido a:

</> Python



▶ Ejecutar

```
[
  {
    "ticker": "TEF",
    "nombre": "Telefónica",
    "precio": 3.5
  },
  {
    "ticker": "CABK",
    "nombre": "CaixaBank",
    "precio": 10.5
  }
]
```

9.

</> Python



▶ Ejecutar

```
conexion.close()
```

Muy importante.

Siempre cerramos la conexión.

Si no, acabaríamos dejando conexiones abiertas y la base de datos terminaría saturándose.

Lo más importante de este paso

Fíjate en algo.

Aquí no aparece Flask.

No aparece HTML.

No aparece CSS.

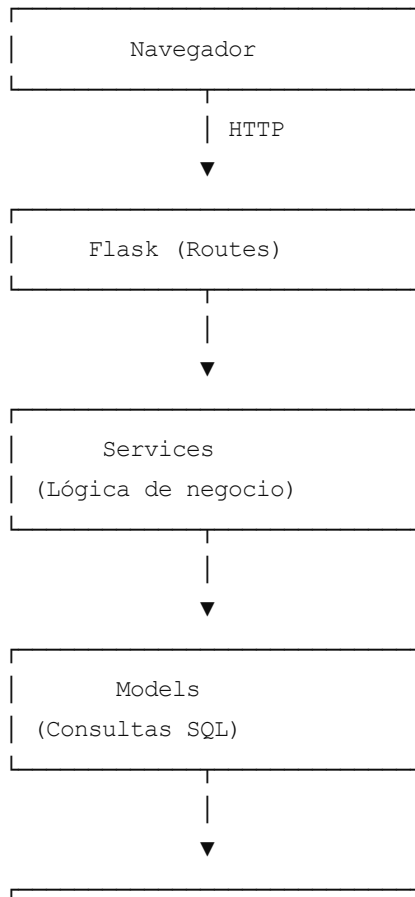
No aparece el navegador.

Este archivo **solo sabe hablar con MySQL**.

Eso es exactamente lo que queremos.

Nuestro objetivo

Vamos a conseguir que la arquitectura quede así:





Un consejo para el TFG

Quiero empezar a introducirte una forma de trabajar que utilizan muchos desarrolladores profesionales: **hacer pequeños commits en Git** cada vez que completamos una funcionalidad importante.

Por ejemplo, después de este paso podrías hacer un commit con el mensaje:

```
Creado modelo de acceso a datos para operaciones de cartera
```



Así tendrás un historial de la evolución del proyecto. Además de ser una buena práctica profesional, demuestra un desarrollo ordenado si alguna vez quieres enseñar el proyecto o subirlo a GitHub.

Tu turno

Crea `models/cartera_model.py` exactamente como hemos visto. Cuando lo tengas, el siguiente paso será emocionante: **hacer que la pantalla "Mi Cartera" lea datos reales desde MySQL y los muestre en el navegador**. Ahí dejará de ser una maqueta y empezará a ser una aplicación funcional.

siguiente

Paso 29: crear el servicio de cartera

Crea este archivo:

```
services/cartera_service.py
```



Pega esto:

<> Python



Ejecutar

```
from models.cartera_model import obtener_operaciones

def obtener_cartera():
    operaciones = obtener_operaciones()

    cartera = []

    for op in operaciones:
        total = op["numero_acciones"] * op["precio"]
```

```

        cargo = total + op["gastos"]

    cartera.append({
        "ticker": op["ticker"],
        "nombre": op["nombre"],
        "numero_acciones": op["numero_acciones"],
        "precio": op["precio"],
        "total": total,
        "gastos": op["gastos"],
        "cargo": cargo,
        "fecha": op["fecha"]
    })

    return cartera

```

¿Qué hace?

El **modelo** trae datos de MySQL.

El **servicio** calcula datos de negocio:

```

total = acciones × precio
cargo = total + gastos

```

Así no mezclamos SQL con cálculos.

Paso 30: usar el servicio en la ruta

Abre:

```
routes/cartera_routes.py
```

Déjalo así:

<> Python



▶ Ejecutar

```

from flask import Blueprint, render_template
from services.cartera_service import obtener_cartera

cartera_bp = Blueprint("cartera", __name__)

@cartera_bp.route("/cartera")
def cartera():
    datos_cartera = obtener_cartera()
    return render_template("cartera.html", cartera=datos_cartera)

```

Ahora la ruta recibe la petición, pide los datos al servicio y se los pasa al HTML.

Cuando lo tengas, seguimos con la tabla en `cartera.html`.

seguimos

Paso 31: mostrar la cartera en pantalla

Abre:

templates/cartera.html



Sustituye su contenido por esto:

HTML



```
{% extends "base.html" %}

{% block title %}GESCAR - Mi Cartera{% endblock %}

{% block content %}

<div class="tarjeta">

    <h2>Mi Cartera</h2>

    <table>
        <thead>
            <tr>
                <th>BME</th>
                <th>Nombre</th>
                <th>Núm</th>
                <th>Importe</th>
                <th>Total</th>
            </tr>
        </thead>
    </table>

</div>

</div>
```

Paso 32: añadir estilo a la tabla

En `static/css/estilos.css`, añade al final:

CSS



```
table{
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}

th{
    background: #003366;
    color: white;
```

```
padding: 10px;
}

td{
padding: 10px;
border-bottom: 1px solid #ddd;
text-align: center;
}

tr:hover{
background: #f0f0f0;
}
```

¿Qué estamos haciendo?

Esta parte:

 HTML



```
{% for accion in cartera %}
```

recorre los datos que vienen desde Python.

Es decir:

MySQL → model → service → route → HTML



Ahora entra en:

http://localhost:5000/cartera



Deberías ver las acciones de prueba.

seguimos

Paso 33: añadir cotización actual y rentabilidad

Ahora vamos a acercar la pantalla al prototipo, donde aparecen:

Valor Actual
Rentabilidad



Abre `models/cartera_model.py` y añade `cotizacion_actual` en el `SELECT` :

<> Python



▶ Ejecutar

```
sql = """
    SELECT
        o.id,
        v.ticker,
        v.nombre,
        v.cotizacion_actual,
        o.numero_acciones,
        o.precio,
        o.gastos,
        o.fecha
    FROM operaciones o
    INNER JOIN valores v
        ON o.valor_id = v.id
    ORDER BY o.fecha
    """
```

Ahora abre `services/cartera_service.py` y déjalo así:

<> Python



▶ Ejecutar

```
from models.cartera_model import obtener_operaciones

def obtener_cartera():
    operaciones = obtener_operaciones()

    cartera = []

    for op in operaciones:
        total = op["numero_acciones"] * op["precio"]
        cargo = total + op["gastos"]

        valor_actual = op["numero_acciones"] * op["cotizacion_actual"]
        rentabilidad = valor_actual - cargo

        if cargo != 0:
            rentabilidad_porcentaje = (rentabilidad / cargo) * 100
        else:
            rentabilidad_porcentaje = 0

    cartera.append({
        "ticker": op["ticker"],
        "nombre": op["nombre"],
        "numero_acciones": op["numero_acciones"],
        "precio": op["precio"],
        "total": total,
        "gastos": op["gastos"],
        "cargo": cargo,
        "cotizacion_actual": op["cotizacion_actual"],
```

```

        "valor_actual": valor_actual,
        "rentabilidad": rentabilidad,
        "rentabilidad_porcentaje": rentabilidad_porcentaje,
        "fecha": op["fecha"]
    })

    return cartera

```

Y en `cartera.html` , añade estas columnas:

HTML

```

<th>Actual</th>
<th>Valor Actual</th>
<th>Rentabilidad</th>
<th>%</th>

```

y dentro del `<tr>` :

HTML

```

<td>{{ accion.cotizacion_actual }} €</td>
<td>{{ accion.valor_actual }} €</td>
<td>{{ accion.rentabilidad }} €</td>
<td>{{ accion.rentabilidad_porcentaje }} %</td>

```

Con esto ya calculamos:

```

valor actual = número de acciones × cotización actual
rentabilidad = valor actual - cargo

```

Prueba `/cartera` y dime si se ven las nuevas columnas.

se ven. He añadido esto en `app.py` para los formatos: #
 Formatos miles y decimales en español
`@app.template_filter('formato_es')`
`def formato_es(valor, decimales=2):`
 `"""Convierte un número a formato español: puntos para`
 `miles, comas para decimales."""`
 `try:`
 `texto = f'{float(valor):.{decimales}f}'`
 `return texto.replace(",", "TEMP").replace(".",`
 `",").replace("TEMP", ",")`
 `except (ValueError, TypeError):`

```
return valor # si no es un número válido, lo devuelve tal cual
```

¡Muy buena idea! De hecho, esta es una mejora que **yo también habría implementado**.



Lo más importante es que **no solo funciona, sino que es una buena práctica**.

¿Qué es un filtro de Jinja?

Recuerda que Flask utiliza el motor de plantillas **Jinja2**.

Hasta ahora hacíamos esto:

```
HTML {{ accion.cargo }}
```

Con el filtro que has creado podemos hacer:

```
HTML {{ accion.cargo | formato_es }}
```

Jinja hace internamente algo parecido a:

```
Python formato_es(accion.cargo)
```

y muestra:

```
12.658,35
```

en lugar de

```
12658.35
```

¿Por qué me gusta esta solución?

Porque has separado responsabilidades.

La lógica de presentación está en el filtro y no repartida por el HTML.

Imagina que dentro de seis meses quieres mostrar 3 decimales.

Con tu sistema cambias un único sitio.

Yo haría una pequeña mejora

En lugar de poner el filtro dentro de `app.py`, lo movería a un archivo independiente.

La estructura sería:

```
GESCAR
|
├── filters
|   ├── __init__.py
|   └── formatos.py
```

`formatos.py`

</> Python



▶ Ejecutar

```
def formato_es(valor, decimales=2):
    try:
        texto = f"{float(valor):.{decimales}f}"
        return texto.replace(",", "TEMP").replace(".", ",").replace("TEMP", ".")
    except (ValueError, TypeError):
        return valor
```

Y en `app.py`:

</> Python



▶ Ejecutar

```
from filters.formatos import formato_es

app.jinja_env.filters["formato_es"] = formato_es
```

¿Por qué?

Porque `app.py` debería ser cada vez más pequeño.

Su función es únicamente arrancar la aplicación.

Incluso podemos mejorarlo más

Como estamos haciendo una aplicación financiera, seguramente necesitaremos varios formatos.

Por ejemplo:

</> Python



Ejecutar

```
{{ accion.cargo | euro }}
```

Mostrará

12.658,35 €



Otro:

</> Python



Ejecutar

```
{{ accion.rentabilidad_porcentaje | porcentaje }}
```

Mostrará

4,76 %



Otro:

</> Python



Ejecutar

```
{{ accion.fecha | fecha_es }}
```

Mostrará

12/03/2026



Y otro:

</> Python



Ejecutar

```
{{ accion.numero_acciones | entero }}
```

Mostrará

15.000



Eso hace que el HTML sea muy limpio:

```
<td>{{ accion.cargo | euro }}</td>

<td>{{ accion.valor_actual | euro }}</td>

<td>{{ accion.rentabilidad | euro }}</td>

<td>{{ accion.rentabilidad_porcentaje | porcentaje }}</td>
```

Fíjate qué diferencia respecto a tener que formatear cada dato manualmente.

Una reflexión importante

Quiero aprovechar este momento para cambiar un poco la forma en que vamos a desarrollar el proyecto.

Hasta ahora hemos ido implementando funcionalidades. A partir de ahora, **vamos a empezar a pensar como arquitectos de software**.

Tu aplicación ya tiene una base sólida. Ahora conviene preguntarnos:

¿Cómo quiero que sea GESCAR cuando esté terminado?

Si no definimos eso desde el principio, corremos el riesgo de ir añadiendo funcionalidades y acabar con un proyecto difícil de mantener.

Mi propuesta (muy importante)

Creo que es el momento de dejar de desarrollar "pantalla a pantalla" y diseñar primero la aplicación completa.

Me gustaría dedicar la siguiente sesión a definir:

- 📁 La estructura definitiva de carpetas.
- 📊 El modelo completo de la base de datos (todas las tablas y relaciones).
- 🔄 El flujo de navegación entre pantallas.
- 🧩 Los módulos que tendrá la aplicación.
- 📐 Un pequeño diagrama de arquitectura que podrás incluir en la memoria.

Después de eso, implementaremos cada módulo siguiendo ese diseño.

¿Por qué? Porque te evitará tener que rehacer código cuando lleguemos a funcionalidades como login, dividendos, simulación de ventas o alarmas. Además, tendrás una memoria mucho más sólida, ya que el diseño de la arquitectura estará pensado desde el principio y no surgirá "sobre la marcha".

🗨 **Mi recomendación es que hagamos ese diseño ahora**, antes de seguir programando. Creo sinceramente que es la decisión que hará que tu TFG pase de ser una aplicación que funciona a un proyecto con una arquitectura profesional.

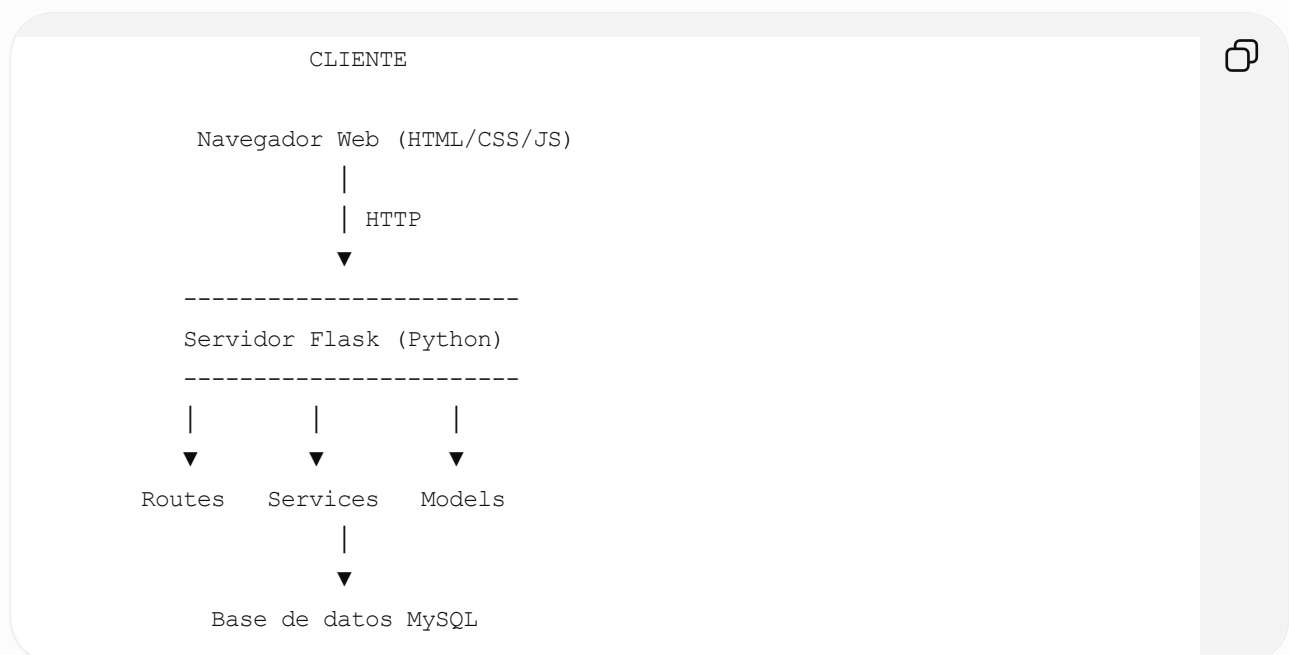
me parece muy bien

Me alegra que estés de acuerdo. De hecho, creo que **este es el mejor momento para hacerlo**. Todavía no hemos escrito demasiado código y estamos a tiempo de construir una aplicación con una arquitectura realmente profesional.

Quiero proponerte algo: **vamos a desarrollar GESCAR como si fuéramos un equipo de software**. No será solo un TFG, sino un proyecto que podrías enseñar en una entrevista de trabajo.

Mi visión para GESCAR

Por el prototipo que me enseñaste y por todo lo que hemos ido desarrollando, yo no haría una simple aplicación web. Haría un **gestor de carteras de inversión** con una arquitectura en tres capas.



Esta arquitectura la podrás explicar perfectamente en la memoria.

Los módulos que tendrá la aplicación

El prototipo tiene cinco módulos, pero creo que deberíamos ampliarlo ligeramente.

1. Login
2. Menú principal
3. Mi cartera
4. Alta de compra
5. Venta / Simulación
6. Histórico
7. Índices
8. Alarmas
9. Perfil de usuario
10. Exportación Excel
11. Informe PDF



Esto da la sensación de una aplicación completa.

El modelo de datos

Aquí es donde quiero hacer un cambio importante respecto a lo que tenemos ahora.

Actualmente tenemos:

```
usuarios
valores
operaciones
```



Es suficiente para hacer pruebas.

Pero no para una aplicación real.

Yo diseñaría esta base de datos

```
USUARIOS
-----
id
```



nombre
usuario
email
telefono
password
fecha_alta

VALORES

id
ticker
nombre
mercado
isin
sector
pais
cotizacion_actual

OPERACIONES

id
usuario_id
valor_id
tipo
numero_acciones
precio
gastos
fecha
comentarios

DIVIDENDOS

id
valor_id
fecha
importe_por_accion

ALARMAS

id
usuario_id
valor_id
tipo
precio_objetivo
activa

INDICES

id
nombre
ticker

COTIZACIONES

id
valor_id
fecha
precio

¿Por qué?

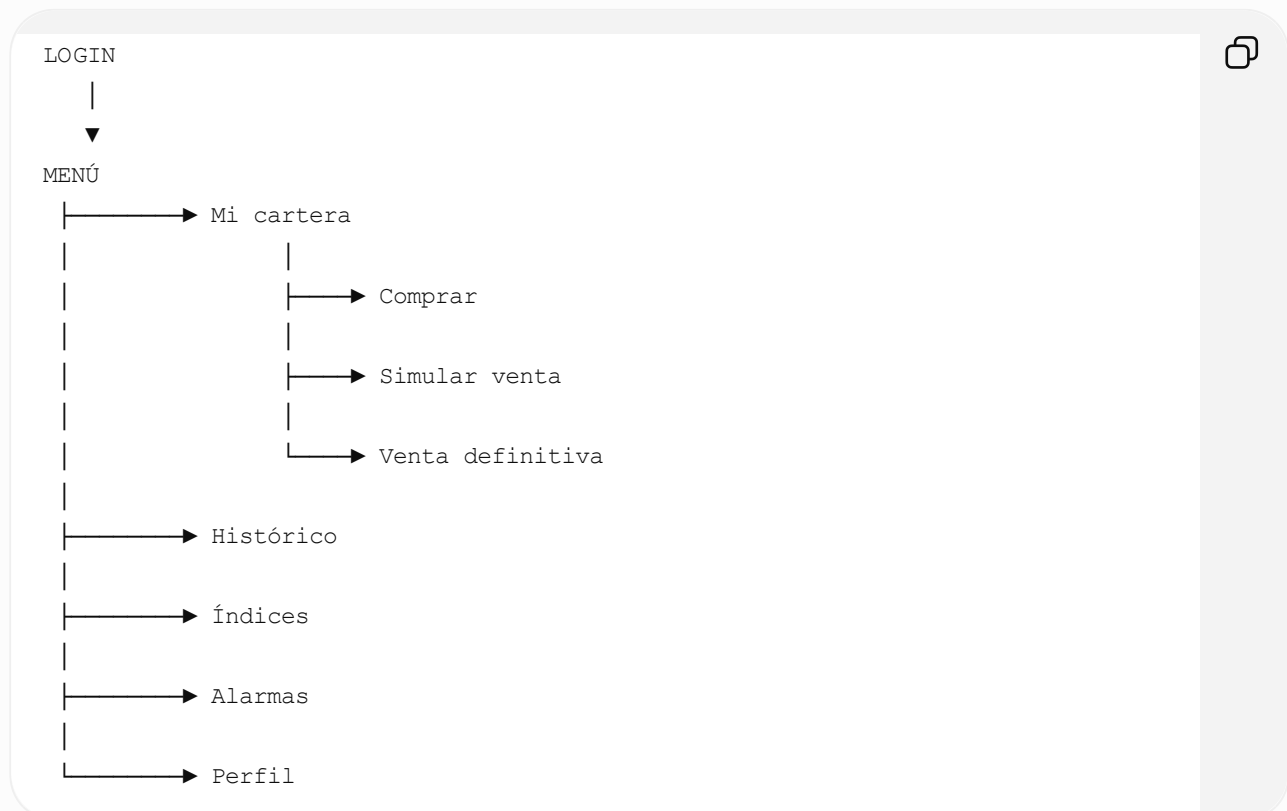
Porque en el futuro podremos:

- calcular dividendos cobrados
- hacer gráficos históricos
- activar alarmas
- mostrar la evolución de las cotizaciones

Sin tener que rehacer la base de datos.

El flujo de navegación

Me gustaría que quedara así:



Así todo tiene sentido.

La estructura definitiva

Yo haría esto:

GESCAR



```
|
├─ app.py
├─ config.py
├─ requirements.txt
|
├─ database
|   ├── conexion.py
|   ├── schema.sql
|   └─ datos_prueba.sql
|
├─ filters
|   └─ formatos.py
|
├─ models
|   ├── usuario_model.py
|   ├── cartera_model.py
|   ├── operacion_model.py
|   ├── alarma_model.py
|   └─ indice_model.py
|
├─ services
|   ├── usuario_service.py
|   ├── cartera_service.py
|   ├── operacion_service.py
|   ├── alarma_service.py
|   └─ pdf_service.py
|
├─ routes
|   ├── login_routes.py
|   ├── cartera_routes.py
|   ├── historico_routes.py
|   ├── alarmas_routes.py
|   ├── altas_routes.py
|   └─ indices_routes.py
|
├─ static
|   ├── css
|   ├── js
|   ├── icons
|   └─ img
|
└─ templates
    ├── base.html
    ├── login.html
    ├── menu.html
    ├── cartera.html
    ├── historico.html
    └─ indices.html
```

Pero hay algo que quiero mejorar aún más

Aquí viene una decisión que creo que marcará la diferencia del proyecto.

Tu prototipo parece una aplicación de escritorio

Las pantallas del PDF tienen una apariencia muy similar a una aplicación Windows.

 Prototipo GESCAR

Si hacemos una web "normal", se parecerá poco al diseño original.

Mi propuesta

En lugar de usar solo HTML y CSS básicos, **utilicemos Bootstrap 5**.

No porque sea "más moderno", sino porque nos aporta:

- Un diseño profesional desde el principio.
- Tablas, formularios y botones con muy buen aspecto.
- Diseño adaptable si algún día quieres verla en una tablet o móvil.
- Mucho menos CSS escrito por nosotros.

Y seguiremos usando HTML, CSS y Flask, simplemente apoyándonos en Bootstrap para acelerar el desarrollo.

Además, quiero añadir un objetivo al proyecto

Quiero que, cuando terminemos, el tribunal piense:

"Esta aplicación podría utilizarse realmente."

Para eso vamos a incluir pequeños detalles profesionales, por ejemplo:

- ☒ Login con contraseñas cifradas (no en texto plano).
- ☒ Validaciones de formularios.
- ☒ Mensajes de éxito y error.
- ☒ Confirmación antes de borrar o vender.
- ☒ Paginación en tablas si hay muchos registros.
- ☒ Búsqueda y filtros.
- ☒ Exportación a Excel y PDF.
- ☒ Código comentado y organizado.

No son difíciles de implementar, pero aportan muchísimo valor al proyecto.

El siguiente paso que te propongo

Ya no será programar una pantalla.

Será **diseñar el corazón de la aplicación**.

Vamos a crear el **modelo entidad-relación (ER)** de la base de datos. No solo nos servirá para implementar correctamente las tablas, sino que también será una figura muy potente para incluir en la memoria del TFG y explicar cómo se relacionan usuarios, operaciones, valores, dividendos y alarmas.

Mi idea es que no solo construyamos la aplicación, sino que generemos durante el desarrollo todo el material que luego necesitarás para redactar la memoria y preparar la defensa. Así, cuando acabemos el software, gran parte del trabajo escrito ya estará prácticamente hecho.